Автономная некоммерческая организация высшего образования
«Московский университет «СИНЕРГИЯ»

Факультет информационных систем и технологий

ОБЗОРНАЯ РЕФЕРАТИВНАЯ РАБОТА

на тему:
«Осознанная интеграция: сборка (building) и тестирование (testing) в CI/CD»

Выполнил: студент
ФИО: Лоор Идальго Жан Пьерр
Группа: ДБО-301рпо
Проверил: _____

Москва, 2026

СОДЕРЖАНИЕ

Введение.....3

1. Понятие осознанной интеграции и ее место в DevOps.....4

2. Сборка программного обеспечения как основа интеграции.....6

3. Тестирование в процессе интеграции.....9

4. Инструменты сборки и тестирования в CI/CD.....12

5. Пример CI-пайплайна для сборки и тестирования.....15

6. Преимущества, риски и практические рекомендации.....17

Заключение.....19

Список использованных источников.....20

Введение

Современная разработка программного обеспечения уже не может рассматриваться как последовательность изолированных этапов, где разработчики долго пишут код, затем отдельно выполняется сборка, а тестирование начинается только перед выпуском продукта. Такой подход приводит к позднему обнаружению ошибок, сложной интеграции отдельных компонентов и увеличению стоимости исправления дефектов. Поэтому в инженерии программного обеспечения особое значение получила практика интеграции изменений на ранних этапах и с высокой регулярностью.

В рамках данной работы термин «осознанная интеграция» используется как акцент на управляемом, дисциплинированном и технически обоснованном подходе к интеграции кода. По своей сути он близок к понятию непрерывной интеграции (Continuous Integration, CI), однако дополнительно подчеркивает необходимость понимания целей каждого действия: зачем выполняется сборка, почему тесты должны запускаться автоматически, какие критерии качества должны блокировать ошибочные изменения и каким образом команда получает обратную связь.

В учебных материалах по CI/CD сборка и тестирование расположены в центральной части процесса: после этапа Code следует Build, а затем Test. С этими этапами связаны такие инструменты, как Gradle, Bazel, Webpack, Jest, JUnit, Jenkins и BuildKit. Это показывает, что сборка и тестирование образуют ядро непрерывной интеграции, а последующие этапы release, deploy, operate и monitor опираются на качество именно этих процессов [2].

В книге П. Дюваля, С. Матиаса и Э. Гловера подчеркивается, что ожидание конца проекта для интеграции и выявления ошибок является неразумным и ведет к снижению качества, удорожанию и задержкам [1].

Следовательно, интеграция должна быть частой, автоматизированной и проверяемой.

Цель работы - рассмотреть осознанную интеграцию как практику организации сборки и тестирования программного обеспечения в CI/CD-процессе.

Для достижения цели были поставлены следующие задачи: раскрыть сущность интеграции в CI/CD; рассмотреть роль сборки как контролируемого процесса получения работоспособного артефакта; описать уровни и виды автоматизированного тестирования; проанализировать инструменты сборки и тестирования; предложить пример CI-пайплайна; сформулировать рекомендации для практического применения.

Объектом исследования является процесс разработки программного обеспечения. Предметом исследования выступают практики сборки и тестирования как ключевые элементы осознанной интеграции.

1. Понятие осознанной интеграции и ее место в DevOps

Интеграция в разработке программного обеспечения означает объединение отдельных изменений, модулей, компонентов и зависимостей в единую работоспособную систему. Чем сложнее проект и чем больше участников работает над кодом, тем выше вероятность конфликтов, несовместимостей и скрытых дефектов. Поэтому интеграция не должна быть редким событием в конце проекта; она должна быть регулярной практикой, встроенной в ежедневную работу команды.

Непрерывная интеграция рассматривается как практика, при которой участники команды часто объединяют свои изменения, обычно не реже одного раза в день, а каждая интеграция сопровождается автоматизированной сборкой

и проверкой. Такой подход позволяет быстро обнаруживать ошибки и уменьшать проблемы, возникающие при объединении кода [1].

Осознанность интеграции проявляется в том, что команда не просто автоматически запускает инструменты, а понимает назначение каждого этапа. Разработчик передает код в репозиторий не как завершенный изолированный фрагмент, а как изменение, которое должно быть проверено в общем контексте системы. CI-сервер, сценарии сборки, автоматизированные проверки и отчеты обратной связи становятся механизмом контроля качества.

В DevOps-подходе интеграция является связующим элементом между разработкой, тестированием и эксплуатацией. Если код регулярно собирается и проверяется, команда получает более высокую уверенность в работоспособности продукта. Это снижает риск ситуации «у меня на машине работает», потому что результат проверяется не только на компьютере разработчика, но и в стандартизированной среде интеграционной сборки.

С точки зрения жизненного цикла CI/CD, осознанная интеграция находится между написанием кода и выпуском продукта. Она включает получение исходного кода из системы контроля версий, установку зависимостей, компиляцию, сборку артефакта, запуск тестов, анализ качества, формирование отчетов и уведомление команды. Только после успешного прохождения этих действий изменение может рассматриваться как пригодное для дальнейшей доставки и развертывания.

Рисунок 1 - Логика осознанной интеграции: сборка, тестирование и обратная связь

Таблица 1 - Основные элементы осознанной интеграции

Элемент интеграции	Назначение	Практический смысл
Хранилище с контролем версий	Единое место хранения исходного кода, сценариев сборки, конфигураций и тестов	Позволяет воспроизвести сборку и отследить источник изменений
CI-сервер	Автоматический запуск pipeline после изменения кода	Исключает ручной запуск проверок и ускоряет обратную связь
Сценарий сборки	Формализованное описание команд сборки и тестирования	Дает возможность выполнить процесс одинаково в любой среде
Тестовый набор	Автоматическая проверка поведения системы	Обнаруживает дефекты до передачи продукта дальше
Отчет обратной связи	Информация о результате сборки, тестов и качества кода	Позволяет быстро исправлять ошибки и не накапливать технический долг

2. Сборка программного обеспечения как основа интеграции

Сборка программного обеспечения (building) - это процесс преобразования исходного кода, ресурсов и зависимостей в готовый программный артефакт. В зависимости от технологии таким артефактом может быть исполняемый файл, архив, библиотека, Docker-образ, пакет frontend-приложения или другой объект, пригодный для тестирования и последующей доставки.

В контексте осознанной интеграции сборка играет роль первого серьезного фильтра качества. Если проект не собирается, то дальнейшее тестирование, релиз или развертывание теряют смысл. Поэтому автоматизированная сборка должна запускаться при каждом значимом изменении кода и давать понятный результат: успешно или неуспешно.

Одна из важнейших практик CI заключается в том, что построение должно выполняться одной командой. Это означает, что разработчик или CI-сервер не должны вручную выполнять десятки отдельных действий. В книге о непрерывной интеграции приводится идея, что все необходимое должно быть получено из хранилища с контролем версий, а вся система должна строиться единой командой [1].

Сборка не должна зависеть от конкретной IDE. Если проект может быть собран только в локальной среде разработчика, то это создает риск невоспроизводимости. Осознанная интеграция требует отделения сценариев сборки от IDE, сохранения конфигураций в репозитории и использования стандартных инструментов сборки.

Важным требованием является скорость сборки. Чем быстрее команда получает ответ, тем быстрее исправляет ошибку. Слишком длительная сборка приводит к тому, что разработчики начинают реже запускать проверки или игнорируют CI. Поэтому практикой является разделение сборок на быстрые и полные: быстрые выполняются при каждом изменении, а более тяжелые тесты и инспекции могут запускаться по расписанию.

Таблица 2 - Практики эффективной сборки в осознанной интеграции

Практика сборки	Содержание	Значение для CI
Автоматизация построения	Сборка запускается скриптом или pipeline, без ручных действий	Снижает риск человеческой ошибки и делает процесс повторяемым

Практика сборки	Содержание	Значение для CI
Одна команда	Весь процесс запускается одной командой, например build или integrate	Упрощает запуск сборки разработчиком и CI-сервером
Независимость от IDE	Сборка не требует открытия проекта в конкретной среде разработки	Позволяет выполнять сборку на выделенной машине
Ранний сбой	Наиболее рискованные операции помещаются ближе к началу pipeline	Команда быстрее получает уведомление о проблеме
Быстрая сборка	Оптимизация времени выполнения, разделение быстрых и длительных проверок	Повышает частоту интеграции и качество обратной связи
Сборка для разных сред	Подготовка артефактов для development, test, staging и production	Помогает выявлять проблемы конфигурации заранее

Сборка должна включать не только компиляцию, но и подготовку зависимостей, выполнение базовых проверок, упаковку артефакта и формирование отчетов. В реальном pipeline она может состоять из этапов clean, install dependencies, compile, lint, unit tests, package и publish artifact.

Для Java-проектов часто применяются Gradle и Maven, для крупных монорепозиториев и сложных зависимостей - Bazel, для frontend-проектов - Webpack или Vite. В приложенных материалах среди источников отдельно выделены Gradle, Bazel и Webpack как технологии, связанные с этапом Build [2].

Осознанная сборка должна быть воспроизводимой. Это означает, что один и тот же исходный код при одинаковых условиях должен давать одинаковый результат. Для этого версии зависимостей фиксируются, конфигурации хранятся в репозитории, а среда сборки стандартизируется. Такой подход позволяет анализировать дефекты не как случайные сбои, а как воспроизводимые проблемы, которые можно исправить.

3. Тестирование в процессе интеграции

Тестирование (testing) в осознанной интеграции необходимо для того, чтобы убедиться: изменение не только успешно собирается, но и сохраняет корректное поведение системы. Если сборка отвечает на вопрос «можно ли получить рабочий артефакт?», то тестирование отвечает на вопрос «работает ли этот артефакт правильно?». Именно поэтому этап Build должен быть связан с этапом Test.

В книге о CI выделяются разные уровни автоматизированных проверок: проверки модуля, компонента, системы и функций. Разделение проверок на категории позволяет организовать CI эффективно: быстрые тесты можно запускать при каждом изменении, а более длительные - позже или по расписанию [1].

Модульные тесты проверяют небольшие фрагменты кода, например отдельные функции или классы. Они должны быть быстрыми, изолированными и не зависеть от внешних систем, таких как база данных или файловая система. Благодаря скорости unit-тесты являются основой быстрого feedback-loop в CI.

Компонентные тесты проверяют взаимодействие нескольких объектов или модулей внутри одного компонента. Они обычно требуют больше времени, чем модульные, но помогают обнаружить ошибки взаимодействия, которые невозможно увидеть на уровне отдельных функций.

Системные тесты проверяют работу нескольких компонентов как единой системы. Они могут включать взаимодействие приложения с базой данных, API, очередями сообщений и другими сервисами. Такие проверки ближе к реальной эксплуатации, но требуют больше ресурсов и времени.

Функциональные и end-to-end тесты оценивают систему с точки зрения пользователя. Для веб-приложений они могут имитировать действия пользователя в браузере. В современных проектах для таких задач могут

использоваться Playwright, Selenium и аналогичные инструменты. В приложенных материалах Playwright указан среди рекомендуемых технологий тестирования [2].

Рисунок 2 - Пирамида автоматизированного тестирования в CI

Таблица 3 - Уровни тестирования в процессе осознанной интеграции

Уровень тестирования	Что проверяет	Особенности запуска в CI
Модульные тесты	Отдельные функции, методы, классы	Запускаются при каждом изменении, должны быть быстрыми и независимыми
Компонентные тесты	Взаимодействие объектов или модулей внутри компонента	Могут запускаться после unit-тестов или в отдельной стадии pipeline
Интеграционные тесты	Работу нескольких компонентов и внешних зависимостей	Требуют подготовленной среды, тестовой базы данных или контейнеров
Системные тесты	Поведение системы как единого продукта	Запускаются дольше, часто используются на nightly/build по расписанию
Функциональные/E2E тесты	Сценарии пользователя и бизнес-функции	Самые дорогие и длительные, применяются для критичных

Уровень тестирования	Что проверяет	Особенности запуска в CI
		пользовательских сценариев

Ключевым принципом является выполнение быстрых проверок раньше медленных. Если ошибка может быть обнаружена модульным тестом за несколько секунд, нет смысла сначала запускать тяжелые системные проверки. Такой порядок экономит ресурсы и ускоряет получение обратной связи.

Не менее важным является принцип написания тестов для обнаруженных дефектов. Если ошибка уже появилась в проекте, после ее исправления следует добавить автоматизированный тест, который не позволит ей повториться незаметно. Это превращает каждый дефект в источник улучшения тестового покрытия.

Тесты должны быть стабильными. Нестабильные проверки, которые иногда проходят, а иногда падают без изменения кода, разрушают доверие к CI. Поэтому в осознанной интеграции важно контролировать тестовую среду, изолировать внешние зависимости, фиксировать тестовые данные и анализировать причины flaky-тестов.

4. Инструменты сборки и тестирования в CI/CD

Инструменты осознанной интеграции можно разделить на несколько групп: системы контроля версий, инструменты сборки, фреймворки тестирования, CI-серверы и средства отчетности. В приложенном документе CI/CD перечислены инструменты, связанные с разными этапами pipeline: GitHub, GitLab, Gradle, Bazel, Webpack, Jest, JUnit, Jenkins, BuildKit и другие [2].

GitHub и GitLab обеспечивают хранение кода, работу с ветками, pull request или merge request, а также могут запускать pipeline. В осознанной интеграции важна не только техническая возможность хранения кода, но и

культура работы: частые коммиты, небольшие изменения, обязательная проверка перед слиянием и прозрачная история изменений.

Gradle является системой автоматизации сборки, особенно распространенной в Java и Kotlin-проектах. Он позволяет описывать задачи сборки, зависимости, тесты и публикацию артефактов. Bazel применяется там, где важны скорость, кэширование и воспроизводимость сборки, особенно в больших проектах. Webpack используется во frontend-разработке для сборки JavaScript, TypeScript, CSS и других ресурсов в готовый пакет приложения.

Jest часто используется для тестирования JavaScript и TypeScript-кода. JUnit является одним из основных фреймворков модульного тестирования для Java. Playwright применяется для браузерного и end-to-end тестирования. Эти инструменты позволяют встроить проверку поведения системы в автоматический pipeline.

Jenkins относится к классическим CI/CD-серверам и позволяет описывать pipeline, запускать сборку и тестирование, подключать плагины, формировать отчеты и уведомлять команду. В современных проектах также используются GitHub Actions, GitLab CI, TeamCity, Bamboo и другие системы, однако смысл остается одинаковым: автоматизировать повторяемые действия и сделать их обязательной частью интеграции.

Таблица 4 - Инструменты сборки и тестирования в CI/CD

Группа	Инструменты	Назначение в осознанной интеграции
Контроль версий	GitHub, GitLab	Хранение кода, работа с ветками, запуск pipeline, code review
Сборка Java/Kotlin	Gradle, Maven	Компиляция, управление зависимостями, запуск тестов, упаковка артефактов
Сборка крупных проектов	Bazel	Быстрая и воспроизводимая сборка, кэширование, работа с большими кодовыми базами
Frontend-сборка	Webpack, Vite	Подготовка frontend-ресурсов, bundling, оптимизация пакетов

Группа	Инструменты	Назначение в осознанной интеграции
Модульное тестирование	JUnit, Jest	Проверка отдельных модулей и функций
E2E-тестирование	Playwright, Selenium	Проверка пользовательских сценариев и поведения интерфейса
CI-сервер	Jenkins, GitLab CI, GitHub Actions	Автоматический запуск сборки, тестов, отчетов и quality gates
Контейнерная сборка	Docker, BuildKit	Создание воспроизводимого образа приложения и тестовой среды

5. Пример CI-пайплайна для сборки и тестирования

Практический смысл осознанной интеграции лучше всего раскрывается через пример pipeline. Рассмотрим типовой процесс для веб-приложения, в котором есть backend, frontend и автоматизированные тесты. Такой pipeline может быть реализован в Jenkins, GitLab CI, GitHub Actions или другой системе.

Процесс начинается с передачи кода в репозиторий. После push или создания merge request CI-система автоматически запускает pipeline. Сначала выполняется получение исходного кода, затем установка зависимостей. После этого запускается сборка backend-части и frontend-части. Если сборка прошла успешно, выполняются модульные тесты. Затем могут запускаться интеграционные и функциональные тесты. После успешной проверки формируется артефакт, например Docker-образ или архив приложения.

Для учебной работы важно показать, что pipeline должен быть не просто набором команд, а системой контроля качества. Каждый этап имеет критерий успешности. Если сборка не выполнена, тесты не запускаются. Если тесты не прошли, артефакт не публикуется. Если покрытие кода ниже установленного порога, изменение блокируется. Именно так автоматизация превращается в осознанную интеграцию.

Листинг 1 - Пример логики CI-пайплайна для сборки и тестирования

```
stages:
  - checkout
  - install
  - build
  - unit_test
  - integration_test
  - quality_gate
  - package

build:
  script:
    - gradle clean build
    - npm install
    - npm run build

unit_test:
  script:
    - gradle test
    - npm run test

integration_test:
  script:
    - docker compose up -d
    - gradle integrationTest
    - npm run e2e

quality_gate:
  script:
    - checkstyle
    - coverage-report

package:
  script:
    - docker build -t application:ci .
```

Таблица 5 - Пример этапов CI-пайплайна

Этап pipeline	Назначение	Критерий успешности
checkout	Получение кода из репозитория	Код и конфигурации успешно загружены
install	Установка зависимостей	Зависимости установлены без конфликтов
build	Компиляция и сборка backend/frontend	Получен рабочий артефакт сборки
unit_test	Запуск быстрых модульных тестов	Все unit-тесты прошли успешно
integration_test	Проверка взаимодействия компонентов	Сервисы корректно работают вместе
quality_gate	Проверка стандартов, покрытия и отчетов	Качество кода соответствует порогам
package	Упаковка артефакта или Docker-образа	Артефакт готов к передаче на следующие этапы

6. Преимущества, риски и практические рекомендации

Главное преимущество осознанной интеграции заключается в раннем обнаружении ошибок. Чем меньше времени проходит между внесением изменения и обнаружением дефекта, тем проще определить причину. Разработчик еще помнит контекст изменения, а список потенциальных причин ограничен небольшим набором последних коммитов.

Второе преимущество - снижение количества ручных операций. Ручная сборка и ручной запуск тестов являются повторяемыми процессами, склонными к ошибкам. Автоматизация позволяет выполнять их одинаково каждый раз и освобождает время команды для анализа, проектирования и разработки.

Третье преимущество - повышение доверия к продукту. Если каждый коммит проходит сборку и тестирование, команда лучше понимает фактическое состояние проекта. Менеджеры получают не только отчеты о выполненных задачах, но и объективные сигналы качества: успешность сборки, результаты тестов, покрытие, количество дефектов и стабильность pipeline.

Однако внедрение осознанной интеграции имеет риски. Первый риск - формальная автоматизация без реального контроля качества. Если pipeline только компилирует код, но не запускает тесты и не анализирует результаты, его ценность ограничена. Второй риск - слишком медленная сборка. Если feedback приходит через час, разработчики теряют мотивацию часто интегрироваться. Третий риск - нестабильные тесты, которые снижают доверие к pipeline.

Для практического применения рекомендуется начинать с малого: автоматизировать сборку одной командой, добавить быстрые unit-тесты, настроить запуск pipeline при каждом merge request, затем постепенно подключать интеграционные и функциональные тесты. Важно не пытаться сразу построить идеальную систему, а развивать pipeline итеративно.

С точки зрения учебной темы, осознанная интеграция может быть представлена как баланс между скоростью и качеством. Быстрая сборка дает оперативную обратную связь, а тестирование подтверждает, что изменения не разрушают систему. Только их совместное применение позволяет получить устойчивый процесс CI.

Таблица 6 - Типовые проблемы и рекомендации

Проблема	Причина	Рекомендация
Сборка занимает слишком много времени	Слишком много длительных тестов в одном pipeline	Разделить проверки на быстрые и полные, применить поэтапную сборку
Pipeline часто падает без причины	Нестабильные тесты или нестабильная среда	Изолировать тесты, фиксировать данные, анализировать flaky-тесты
Разработчики игнорируют CI	Нет ответственности за сломанную сборку	Ввести правило немедленного исправления failed build
Сборка работает только локально	Зависимость от IDE или локальных настроек	Хранить сценарии сборки и конфигурации в репозитории
Тестов много, но они бесполезны	Нет связи с реальными дефектами и требованиями	Писать тесты для обнаруженных дефектов и критичных сценариев

Заключение

В ходе выполнения обзорной реферативной работы была рассмотрена тема осознанной интеграции как подхода к организации сборки и тестирования программного обеспечения в CI/CD-процессе. Было показано, что интеграция не должна восприниматься как редкое событие в конце проекта. Напротив, она должна выполняться часто, автоматически и с понятной обратной связью.

Сборка является первым ключевым элементом осознанной интеграции. Она позволяет проверить, может ли исходный код быть преобразован в рабочий программный артефакт. Для эффективной сборки необходимо использовать автоматизацию, запуск одной командой, независимость от IDE, хранение сценариев в системе контроля версий и стремление к быстрому получению результата.

Тестирование является вторым ключевым элементом интеграции. Оно подтверждает, что собранный продукт работает корректно. В работе были рассмотрены модульные, компонентные, интеграционные, системные и функциональные тесты. Особое внимание уделено принципу запуска быстрых проверок раньше медленных, а также необходимости писать тесты для обнаруженных дефектов.

В работе также были проанализированы инструменты, применяемые в CI/CD: GitHub, GitLab, Gradle, Bazel, Webpack, Jest, JUnit, Playwright, Jenkins, Docker и BuildKit. Их использование позволяет автоматизировать повторяемые операции, снизить риск человеческих ошибок и сделать процесс разработки более прозрачным.

Таким образом, цель работы достигнута: рассмотрены теоретические основы и практическая логика осознанной интеграции, раскрыта роль сборки и тестирования как центральных этапов CI, а также предложен пример pipeline, демонстрирующий применение этих принципов в современной разработке программного обеспечения.

Список использованных источников

1. Дюваль П. М., Матиас С., Гловер Э. Непрерывная интеграция: улучшение качества программного обеспечения и снижение риска. - М.: Вильямс, 2008. - 240 с.

2. Fowler M. Continuous Integration. URL: https://martinfowler.com/articles/continuousIntegration.html

3. CICD: подборка материалов по Gradle, Bazel, Webpack, Jest, JUnit, Jenkins, BuildKit, Prometheus, Terraform, Kubernetes, Argo CD. Материалы из приложения к заданию.

4. Gradle User Manual. URL: https://docs.gradle.org/current/userguide/userguide.html

5. Bazel Documentation. URL: https://bazel.build/docs

6. Webpack Documentation. URL: https://webpack.js.org/concepts/

7. Jest Documentation. URL: https://jestjs.io/docs/getting-started

8. JUnit 5 User Guide. URL: https://junit.org/junit5/docs/current/user-guide/

9. Playwright Documentation. URL: https://playwright.dev/docs/intro

10. Jenkins User Documentation. URL: https://www.jenkins.io/doc/

11. GitHub Actions Documentation. URL: https://docs.github.com/actions

12. GitLab CI/CD Documentation. URL: https://docs.gitlab.com/ci/

13. Docker Build Documentation. URL: https://docs.docker.com/build/

14. BuildKit в Kubernetes: мануал по быстрой и автомасштабируемой сборке проектов // Хабр. URL: https://habr.com/ru/companies/kts/articles/960922/

15. Основы CI/CD. Знакомство с Jenkins // Хабр. URL: https://habr.com/ru/companies/slurm/articles/691876/

16. Gradle - система автоматизации сборки Java-проектов // Хабр. URL: https://habr.com/ru/hubs/gradle/posts/

17. Сборка с Bazel в реальном проекте // Хабр. URL: https://habr.com/ru/companies/ntechlab/articles/540728/

18. Webpack: руководство для начинающих // Хабр. URL: https://habr.com/ru/articles/514838/